

FASE 2: MACHINE LEARNING PREDITIVO E AVALIAÇÃO

Objetivo: Dominar todo o ciclo de vida do Machine Learning clássico — da engenharia de atributos (*feature engineering*) à modelagem supervisionada/não supervisionada e ao diagnóstico minucioso do desempenho do modelo.

Módulo 4: Machine Learning Essencial

1. Teoria Aprofundada

A. Regressão (Supervisionado - Variável Contínua)

- **Regressão Lineal Simples e Múltipla:**

Modela a relação entre variáveis preditoras $\mathbf{X} \in \mathbb{R}^{n \times d}$ e uma variável alvo contínua $y \in \mathbb{R}$:

$$\hat{y} = \mathbf{X}\mathbf{w} + b$$

Otimizado minimizando a Soma dos Erros Quadráticos (Ordinary Least Squares - OLS):

$$J(\mathbf{w}) = \frac{1}{2n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- **Regularização (Prevenção de Overfitting):**

- **Ridge (L_2):** Adiciona a penalidade $\|\mathbf{w}\|_2^2$ ao custo. Mantém todos os coeficientes, mas reduz suas magnitudes.
- **Lasso (L_1):** Adiciona a penalidade $\|\mathbf{w}\|_1$. Realiza seleção de atributos ao zerar coeficientes irrelevantes.
- **ElasticNet:** Combina penalidades L_1 e L_2 .

B. Classificação (Supervisionado - Variável Categórica)

- **Regressão Logística:**

Aplica a função Sigmoid $\sigma(z) = \frac{1}{1+e^{-z}}$ à saída linear para estimar a probabilidade $P(y = 1|\mathbf{x})$. Otimizado via *Binary Cross-Entropy Loss* (Log-Loss).

- **Árvores de Decisão e Ensemble Methods:**

- **Árvores de Decisão:** Dividem o espaço de atributos recursivamente com base em critérios de pureza:
 - **Impureza de Gini:** $G = 1 - \sum_{k=1}^K p_k^2$
 - **Entropia / Ganho de Informação:** $H(S) = - \sum_{k=1}^K p_k \log_2(p_k)$
- **Random Forest (Bagging):** Combina múltiplas árvores de decisão treinadas em subamostras de dados (bootstrapping) e

subconjuntos aleatórios de atributos, reduzindo a variância do modelo.

- **Gradient Boosting / XGBoost / LightGBM (Boosting):** Constrói árvores de forma sequencial, onde cada nova árvore é treinada para corrigir os resíduos (erros) das árvores anteriores.

C. Clustering (Não Supervisionado)

- **K-Means:** Agrupa dados em K clusters minimizando a inércia (Soma dos Quadrados Intra-cluster - WCSS):

$$WCSS = \sum_{k=1}^K \sum_{\mathbf{x} \in C_k} \|\mathbf{x} - \boldsymbol{\mu}_k\|^2$$

Determina-se o K ideal via Método do Cotovelo (*Elbow Method*) e Pontuação de Silhueta (*Silhouette Score*).

- **DBSCAN (Density-Based Spatial Clustering of Applications with Noise):** Agrupa pontos em regiões de alta densidade definidos por ϵ (raio) e $min_samples$. É capaz de identificar *outliers* (ruído) e formar clusters de formatos arbitrários.

D. Feature Engineering & Pré-Processamento

- **Dimensionamento:** *StandardScaler* (Z-Score: $\frac{x-\mu}{\sigma}$) vs. *MinMaxScaler* (Escala min-max $[0,1]$).
- **Codificação Categórica:** *One-Hot Encoding* para dados nominais e *Ordinal Encoding* para dados ordinais.
- **Seleção e Extração de Atributos:** Importância de atributos (Gini Importance/SHAP) e Análise de Componentes Principais (PCA) para redução de dimensionalidade.

2. Prática Mão na Massa (Python / Scikit-Learn)

```
import numpy as np
import pandas as pd
from sklearn.datasets import make_classification, make_blobs
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier
from sklearn.cluster import KMeans

# 1. Criação de um Dataset Sintético
X_raw, y_raw = make_classification(
    n_samples=1000, n_features=4, n_informative=3, n_redundant=1,
    random_state=42
```

```

)

df = pd.DataFrame(X_raw, columns=['feat_num_1', 'feat_num_2',
'feat_num_3', 'feat_num_4'])
# Adicionando uma coluna categórica artificial
df['categoria'] = np.random.choice(['Baixo', 'Medio', 'Alto'], size=1000)

# 2. Pipeline de Pré-Processamento (Feature Engineering)
num_cols = ['feat_num_1', 'feat_num_2', 'feat_num_3', 'feat_num_4']
cat_cols = ['categoria']

preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), num_cols),
        ('cat', OneHotEncoder(drop='first'), cat_cols)
    ]
)

# 3. Pipeline Completo de Classificação (Random Forest)
pipeline_rf = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('classifier', RandomForestClassifier(n_estimators=100,
random_state=42))
])

X_train, X_test, y_train, y_test = train_test_split(df, y_raw,
test_size=0.2, random_state=42)

# Treino e Predição
pipeline_rf.fit(X_train, y_train)
y_pred_rf = pipeline_rf.predict(X_test)
print("Pipeline de Classificação treinado e testado com sucesso.")

# 4. Exemplo de Clustering Não Supervisionado (K-Means)
X_cluster, _ = make_blobs(n_samples=500, centers=3, n_features=2,
random_state=42)
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
cluster_labels = kmeans.fit_predict(X_cluster)

print(f"Centroides dos Clusters Encontrados:\n{kmeans.cluster_centers_}")

```

Módulo 5: Avaliação de Modelos

1. Teoria

A. Matriz de Confusão e Métricas de Classificação

- **Verdadeiro Positivo (TP), Falso Positivo (FP), Verdadeiro Negativo (TN), Falso Negativo (FN).**
- **Acurácia (Accuracy):** $\frac{TP+TN}{TP+TN+FP+FN}$ (Enganosa em datasets desbalanceados).
- **Precisão (Precision):** $\frac{TP}{TP+FP}$ — De tudo o que o modelo previu como positivo, quanto realmente é positivo? (Crucial para conter Falsos Alarmes/Spam).
- **Revocação / Sensibilidade (Recall):** $\frac{TP}{TP+FN}$ — De tudo o que é realmente positivo, quanto o modelo conseguiu capturar? (Crucial para Diagnósticos Médicos e Detecção de Fraude).
- **F1-Score:** Média harmônica entre Precisão e Recall:

$$F1 = 2 \times \frac{\text{Precisão} \times \text{Recall}}{\text{Precisão} + \text{Recall}}$$

- **Curva ROC e AUC (Area Under Curve):** Plota a Taxa de Verdadeiros Positivos ($TPR = \text{Recall}$) contra a Taxa de Falsos Positivos ($FPR = \frac{FP}{FP+TN}$) em vários limiares (*thresholds*).
 - $AUC = 1.0$: Classificador perfeito.
 - $AUC = 0.5$: Classificador aleatório.

B. Métricas de Regressão

- **Erro Médio Absoluto (MAE):** $\frac{1}{n} \sum |y_i - \hat{y}_i|$ (Lida bem com *outliers*).
- **Erro Quadrático Médio (MSE):** $\frac{1}{n} \sum (y_i - \hat{y}_i)^2$ (Penaliza erros grandes de forma quadrática).
- **Raiz do Erro Quadrático Médio (RMSE):** $\sqrt{MSE}$ (Na mesma unidade da variável alvo).
- **Coeficiente de Determinação (R^2):** Proporção da variância explicada pelo modelo.

2. Prática Mão na Massa (Python / Metrics)

```
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    roc_auc_score, confusion_matrix, classification_report,
    mean_absolute_error, mean_squared_error, r2_score
)

# 1. Avaliação do Modelo de Classificação treinado no Módulo anterior
y_probs_rf = pipeline_rf.predict_proba(X_test)[: , 1]
```

```

acc = accuracy_score(y_test, y_pred_rf)
prec = precision_score(y_test, y_pred_rf)
rec = recall_score(y_test, y_pred_rf)
f1 = f1_score(y_test, y_pred_rf)
auc = roc_auc_score(y_test, y_probs_rf)

print("=== MÉTRICAS DE CLASSIFICAÇÃO ===")
print(f"Acurácia: {acc:.4f}")
print(f"Precisão: {prec:.4f}")
print(f"Recall: {rec:.4f}")
print(f"F1-Score: {f1:.4f}")
print(f"ROC AUC: {auc:.4f}")
print("\nMatriz de Confusão:")
print(confusion_matrix(y_test, y_pred_rf))

# 2. Simulação de Avaliação de Regressão
y_real_reg = np.array([100.0, 150.0, 200.0, 250.0, 300.0])
y_pred_reg = np.array([105.0, 142.0, 208.0, 240.0, 315.0])

mae = mean_absolute_error(y_real_reg, y_pred_reg)
rmse = np.sqrt(mean_squared_error(y_real_reg, y_pred_reg))
r2 = r2_score(y_real_reg, y_pred_reg)

print("\n=== MÉTRICAS DE REGRESSÃO ===")
print(f"MAE: {mae:.2f}")
print(f"RMSE: {rmse:.2f}")
print(f"R²: {r2:.4f}")

```

FASE 3: INTRODUÇÃO À IA GENERATIVA E COMUNICAÇÃO COM LLMS

Objetivo: Compreender a arquitetura interna e limitações dos Grandes Modelos de Linguagem (LLMs) e dominar as estratégias de Engenharia de Prompts para guiar o comportamento dessas redes.

Módulo 6: GenAI e LLMs

1. Teoria Aprofundada

A. Tokenização e Espaço do Contexto

- **Tokens:** A unidade fundamental de processamento de um LLM. Um token pode ser uma palavra, subpalavra (*subword*) ou caractere. Modelos como GPT usam *Byte-Pair Encoding* (BPE) ou *Tiktoken*.
- **Janela de Contexto (*Context Window*):** O limite máximo de tokens que o modelo consegue processar simultaneamente (incluindo prompt de entrada e resposta). É delimitado pelo mecanismo de autoatenção (*Self-Attention*) da arquitetura *Transformer* ($O(N^2)$ em memória).

B. Grounding, Alucinação e Limitações

- **Alucinação (*Hallucination*):** Ocorre quando o modelo gera respostas plausíveis do ponto de vista gramatical e sintático, porém faticamente incorretas ou inventadas, devido à natureza probabilística de prever o próximo token (*Next-Token Prediction*).
- **Grounding (Ancoragem):** Técnica de fornecer dados reais, atualizados ou documentos de contexto dentro do prompt para fundamentar e delimitar a resposta do LLM, mitigando alucinações.
- **Limitações Inerentes:**
 - Falta de acesso estático a dados em tempo real após a data de corte do treino (*Knowledge Cutoff*).
 - Dificuldade com cálculos matemáticos exatos sem execução de código externo.

2. Prática Mão na Massa (Python / Tiktoken / Simulação de Grounding)

```
import tiktoken

# 1. Demonstração de Tokenização
encoder = tiktoken.get_encoding("cl100k_base") # Encoder padrão do GPT-4

texto = "Engenharia de Prompts e RAG são pilares da IA Generativa."
tokens = encoder.encode(texto)

print("=== TOKENIZAÇÃO ===")
print(f"Texto Original: '{texto}'")
print(f"Quantidade de Tokens: {len(tokens)}")
print(f"IDs dos Tokens: {tokens}")
print("Tokens Decodificados Individualmente:")
for t in tokens:
    print(f"  ID {t} -> '{encoder.decode([t])}'")

# 2. Exemplo Conceitual de Grounding para Prevenção de Alucinação
def responder_com_grounding(pergunta: str, documento_contexto: str) -> str:
    prompt_ancorado = f"""
Você é um assistente estritamente factual. Responda à pergunta baseando-se EXCLUSIVAMENTE no contexto fornecido abaixo.
Se a informação não estiver contida no contexto, responda: "Não possui informações suficientes para responder."

--- CONTEXTO ---
{documento_contexto}

--- PERGUNTA ---
{pergunta}
"""
    return prompt_ancorado
```

```
contexto_empresa = "A empresa TechCorp foi fundada em 2018 e atua no  
setor de computação quântica."  
prompt_gerado = responder_com_grounding("Qual é o faturamento da TechCorp  
em 2025?", contexto_empresa)  
  
print("\n=== PROMPT ANCORADO (GROUNDING) ===")  
print(prompt_gerado)
```

Módulo 7: Engenharia de Prompts

1. Teoria Aprofundada

A. Padrões de Prompting (Prompt Patterns)

- **System Prompt vs. User Prompt:**
 - *System Prompt:* Define a persona, diretrizes de segurança, tom de voz e restrições globais do modelo.
 - *User Prompt:* A instrução ou pergunta direta enviada pelo usuário.
- **Zero-Shot Prompting:** Pedir a execução de uma tarefa sem nenhum exemplo prévio.
- **Few-Shot Prompting:** Fornecer ao modelo alguns exemplos de pares (entrada → saída) para demonstrar o formato e o padrão desejados.
- **Chain-of-Thought (CoT - Cadeia de Pensamento):** Instruir o modelo a explicitar o raciocínio passo a passo ("*Vamos pensar passo a passo*"), aumentando dramaticamente a precisão em tarefas de lógica e matemática.

B. Decomposição de Tarefas e Prompts Técnicos

- **Decomposição (Task Decomposition):** Quebrar problemas complexos em sub-prompts encadeados (ex.: 1. Extrair entidades → 2. Analisar sentimento → 3. Gerar relatório formatado em JSON).
- **Saídas Estruturadas (Structured Outputs):** Forçar o LLM a retornar dados no formato JSON Schema, garantindo integração segura com APIs e aplicações backend.

2. Prática Mão na Massa (Prompt Engineering)

```
import json  
  
# 1. Padrão Few-Shot + Cadeia de Pensamento (Chain-of-Thought)  
prompt_cot_fewshot = """  
Você é um analisador financeiro. Determine se a transação é suspeita.  
Pense passo a passo.
```

Exemplo 1:

Transação: R\$ 50,00 em supermercado local às 14h.

Raciocínio: O valor é baixo, o local é comum e o horário é comercial.

Resultado: {"suspeita": false, "risco": "Baixo"}

Exemplo 2:

Transação: R\$ 45.000,00 em loja de joias às 03h da manhã em outro país.

Raciocínio: Valor extremamente elevado, horário atípico e localização geográfica divergente.

Resultado: {"suspeita": true, "risco": "Alto"}

--- TAREFA ---

Transação: R\$ 12.000,00 em eletrônicos às 02h15min.

Raciocínio:

"""

```
print("=== PROMPT FEW-SHOT + CHAIN OF THOUGHT ===")
```

```
print(prompt_cot_fewshot)
```

2. Template de Prompt para Saída JSON Estrita (Técnico)

```
def criar_prompt_extracao_json(texto_bruto: str) -> str:
```

```
    schema = {
        "nome_cliente": "string",
        "produto_mencionado": "string",
        "nivel_insatisfacao": "int (1 a 5)"
    }
```

```
    prompt = f"""
```

Extraia as informações do texto do cliente e responda APENAS com um objeto JSON válido seguindo este esquema:

```
{json.dumps(schema, indent=2)}
```

Não inclua textos explicativos adicionais nem marcações de código markdown além do JSON.

Texto do Cliente:

```
"{texto_bruto}"
```

```
"""
```

```
    return prompt
```

```
print("\n=== PROMPT PARA SAÍDA ESTRUTURADA (JSON) ===")
```

```
print(criar_prompt_extracao_json("Olá, meu nome é Carlos. Comprei o notebook modelo X100 mas a tela veio quebrada! Nota 5 de insatisfação!"))
```